

Server web server with other services

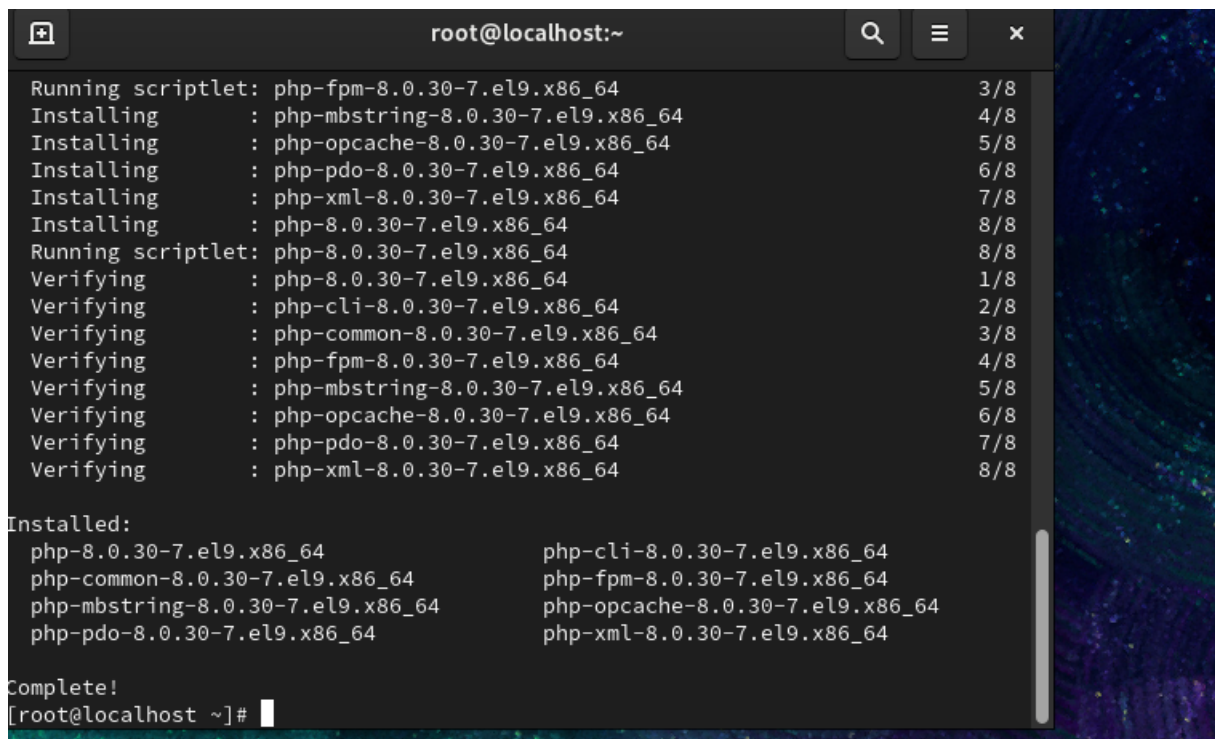
TASKS

1. Install Apache web server and PHP on your Linux machine, then create an index.php file that displays 'Welcome to My Foodie Blog' when accessed via your browser.

Install Apache + PHP

Step 1: Install Apache and PHP

`dnf install httpd php php-cli -y`



```
root@localhost:~  
Running scriptlet: php-fpm-8.0.30-7.el9.x86_64 3/8  
Installing      : php-mbstring-8.0.30-7.el9.x86_64 4/8  
Installing      : php-opcache-8.0.30-7.el9.x86_64 5/8  
Installing      : php-pdo-8.0.30-7.el9.x86_64 6/8  
Installing      : php-xml-8.0.30-7.el9.x86_64 7/8  
Installing      : php-8.0.30-7.el9.x86_64 8/8  
Running scriptlet: php-8.0.30-7.el9.x86_64 8/8  
Verifying       : php-8.0.30-7.el9.x86_64 1/8  
Verifying       : php-cli-8.0.30-7.el9.x86_64 2/8  
Verifying       : php-common-8.0.30-7.el9.x86_64 3/8  
Verifying       : php-fpm-8.0.30-7.el9.x86_64 4/8  
Verifying       : php-mbstring-8.0.30-7.el9.x86_64 5/8  
Verifying       : php-opcache-8.0.30-7.el9.x86_64 6/8  
Verifying       : php-pdo-8.0.30-7.el9.x86_64 7/8  
Verifying       : php-xml-8.0.30-7.el9.x86_64 8/8  
  
Installed:  
php-8.0.30-7.el9.x86_64      php-cli-8.0.30-7.el9.x86_64  
php-common-8.0.30-7.el9.x86_64  php-fpm-8.0.30-7.el9.x86_64  
php-mbstring-8.0.30-7.el9.x86_64  php-opcache-8.0.30-7.el9.x86_64  
php-pdo-8.0.30-7.el9.x86_64      php-xml-8.0.30-7.el9.x86_64  
  
Complete!  
[root@localhost ~]#
```

Step 2: Start Apache

`systemctl enable httpd`
`systemctl start httpd`

Step 3: Check Status

`systemctl status httpd`

Expected:

Active: active (running)

```
[root@localhost ~]# systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Drop-In: /usr/lib/systemd/system/httpd.service.d
            └─php-fpm.conf
   Active: active (running) since Fri 2026-07-17 12:36:42 IST; 54min ago
     Docs: man:httpd.service(8)
  Main PID: 10093 (httpd)
    Status: "Total requests: 1; Idle/Busy workers 100/0;Requests/sec: 0.000304"
     Tasks: 177 (limit: 4273)
    Memory: 6.9M (peak: 15.2M)
       CPU: 4.497s
    CGroup: /system.slice/httpd.service
            └─10093 /usr/sbin/httpd -DFOREGROUND
              └─10094 /usr/sbin/httpd -DFOREGROUND
                └─10096 /usr/sbin/httpd -DFOREGROUND
                  └─10099 /usr/sbin/httpd -DFOREGROUND
                    └─10101 /usr/sbin/httpd -DFOREGROUND

Jul 17 12:36:42 localhost.localdomain systemd[1]: Starting The Apache HTTP Serv
Jul 17 12:36:42 localhost.localdomain httpd[10093]: AH00558: httpd: Could not r
Jul 17 12:36:42 localhost.localdomain systemd[1]: Started The Apache HTTP Serve
Jul 17 12:36:42 localhost.localdomain httpd[10093]: Server configured, listenin
lines 1-22/22 (END)
```

**Step 4:
Create**

index.php

cd /var/www/html

vi index.php

Press i

Paste:

<?php

echo "<h1>Welcome to My Foodie Blog</h1>";

?>

Save

Esc

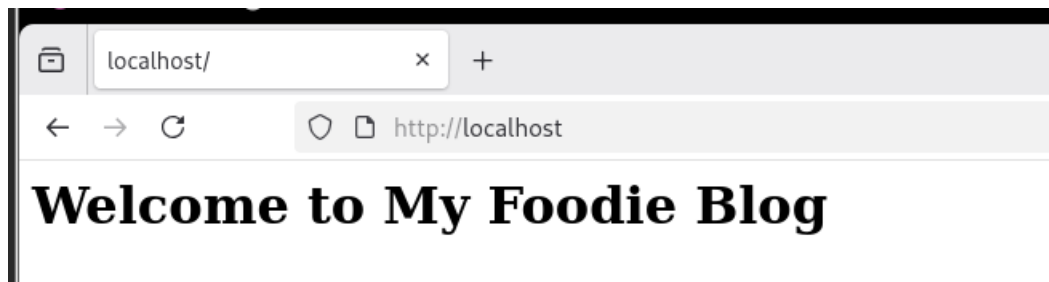
:wq

Step 5: Remove index.html (if present)

rm -f /var/www/html/index.html

Step 6: Open Browser

http://localhost



2. Configure your Apache server to serve a custom 404 error page that displays a Zomato-style message: 'Oops! This page is as empty as your cart.'

Custom 404 Page

Step 1: Create Error Page

```
vi /var/www/html/404.html
```

Paste

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>404</title>
```

```
</head>
```

```
<body>
```

```
<h1>Oops!</h1>
```

```
<h2>This page is as empty as your cart.</h2>
```

```
</body>
```

```
</html>
```

Save

Esc

:wq

Step 2: Configure Apache

`vi /etc/httpd/conf/httpd.conf`

Go to the end

Add

`ErrorDocument 404 /404.html`

Save

Esc

`:wq`

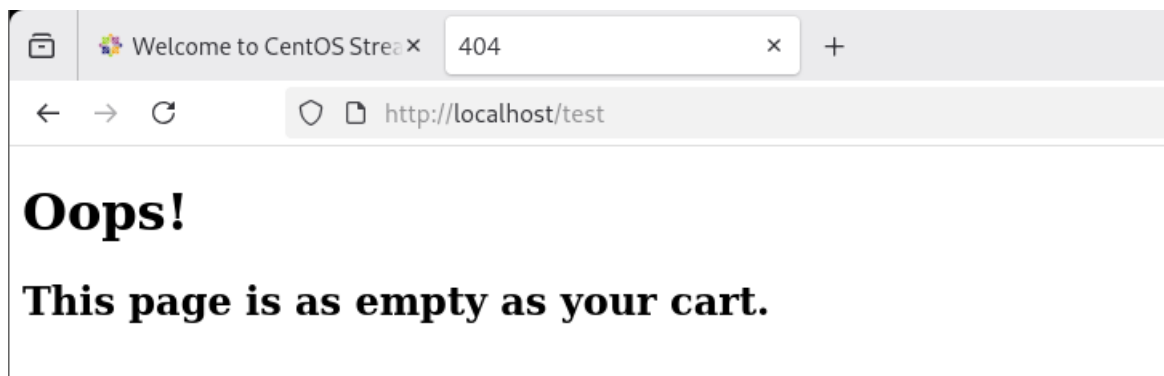
Step 3: Restart Apache

`systemctl restart httpd`

Step 4: Test

Open

`http://localhost/test`



3. Set up a simple FTP server (vsftpd or any other) on your Linux system and upload a sample image file to your web server's root directory using an FTP client.

FTP Server

Step 1: Install FTP

`dnf install vsftpd -y`

Step 2: Start FTP

```
systemctl enable vsftpd
systemctl start vsftpd
```

Step 3: Check Status

```
systemctl status vsftpd
```

```
Complete:
[root@localhost html]# systemctl enable vsftpd
Created symlink /etc/systemd/system/multi-user.target.wants/vsftpd.service → /usr/lib/systemd/system/vsftpd.service.
[root@localhost html]# systemctl start vsftpd
[root@localhost html]# systemctl status vsftpd
● vsftpd.service - Vsftpd ftp daemon
   Loaded: loaded (/usr/lib/systemd/system/vsftpd.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-07-17 13:47:33 IST; 19s ago
     Process: 15107 ExecStart=/usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf (code=exited, status=0/SUCCESS)
    Main PID: 15108 (vsftpd)
      Tasks: 1 (limit: 4273)
     Memory: 1.0M (peak: 1.4M)
        CPU: 5ms
    CGroup: /system.slice/vsftpd.service
            └─15108 /usr/sbin/vsftpd /etc/vsftpd/vsftpd.conf

Jul 17 13:47:33 localhost.localdomain systemd[1]: Starting Vsftpd ftp daemon...
Jul 17 13:47:33 localhost.localdomain systemd[1]: Started Vsftpd ftp daemon.
lines 1-13/13 (END)
```

Step
4:
Allow
FTP

```
firewall-cmd --permanent --add-service=ftp
```

```
firewall-cmd --reload
```

Step 5: Go to Web Directory

```
cd /var/www/html
```

Step 6: Create a Sample Image File

```
touch sample.jpg
```

Step 7: Verify

```
ls -l /var/www/html
```

```
ls /var/www/html
```

```
[root@localhost html]# touch sample.jpg
[root@localhost html]# ls -l /var/www/html
total 8
-rw-r--r--. 1 root root 143 Jul 17 13:44 404.html
drwxr-xr-x. 3 root root 20 Jul 14 14:27 foodie
-rw-r--r--. 1 root root 52 Jul 17 13:35 index.php
drwxr-xr-x. 2 root root 43 Jul 14 12:04 myflipkart
drwxr-xr-x. 2 apache apache 24 Jul 14 13:45 mysite
-rw-r--r--. 1 root root 0 Jul 17 13:51 sample.jpg
-rw-r--r--. 1 root root 0 Jul 14 14:19 testfile.txt
```

4. Configure your web server to also run a basic Node.js service on a different port (e.g., 3000) that returns JSON data for a mock 'Now Playing'

playlist, similar to Spotify. Test accessing both the PHP web page and the Node.js playlist endpoint from your browser.

Node.js Service

Step 1: Install Node.js

```
dnf module install nodejs:20 -y
```

Verify

```
node -v
```

Step 2: Create Folder

```
mkdir /root/nodeapp
```

```
cd /root/nodeapp
```

Step 3: Create App

```
vi app.js
```

Paste

```
const http = require('http');

http.createServer((req,res)=>{

res.writeHead(200,{'Content-Type':'application/json'});

res.end(JSON.stringify({

playlist:"Now Playing",

song:"Perfect"

}));

}).listen(3000);

console.log("Server running on port 3000");
```

Save

Esc
:wq

Step 4: Run

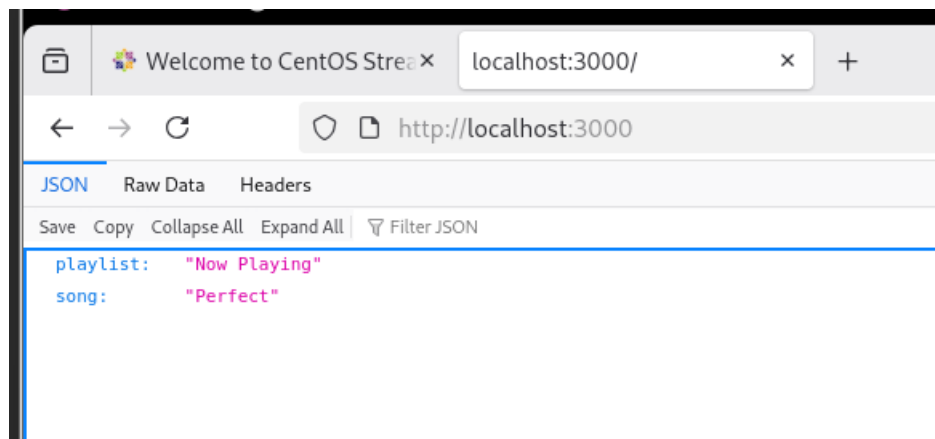
node app.js

```
[root@localhost nodeapp]# vi app.js
[root@localhost nodeapp]# node app.js
Server running on port 3000
```

Step 5: Test

Open browser

http://localhost:3000



5. Edit your Apache configuration to restrict access to a specific directory (e.g., /var/www/html/secret) so that only users from localhost (127.0.0.1) can access it. Hint: Use the 'Require' directive in your Apache config or .htaccess file.

Restrict Access

Step 1: Create Secret Folder

```
mkdir /var/www/html/secret
```

Step 2: Create File

```
vi /var/www/html/secret/index.html
```

Paste

```
<h1>Top Secret Data</h1>
```

Save

Esc

:wq

Step 3: Configure Apache

vi /etc/httpd/conf/httpd.conf

Go bottom

Add

<Directory "/var/www/html/secret">

Require ip 127.0.0.1

</Directory>

Save

Esc

:wq

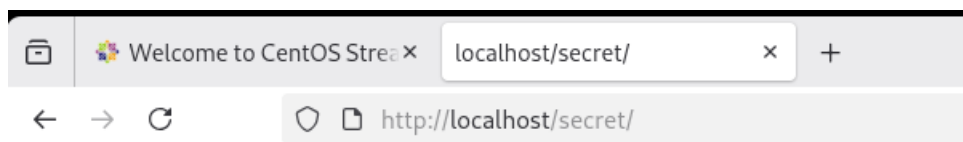
Step 4: Restart Apache

systemctl restart httpd

Step 5: Test

From localhost

http://localhost/secret



Top Secret Data